@aman_views

TOP 30 KAFKA INTERVIEW QUESTIONS

1. What is Apache Kafka and why is it used?

Apache Kafka is a distributed event streaming platform designed to handle high-throughput, real-time data feeds. It is used for building real-time data pipelines, streaming applications, log aggregation, event sourcing, messaging systems, and more.

Why Kafka?

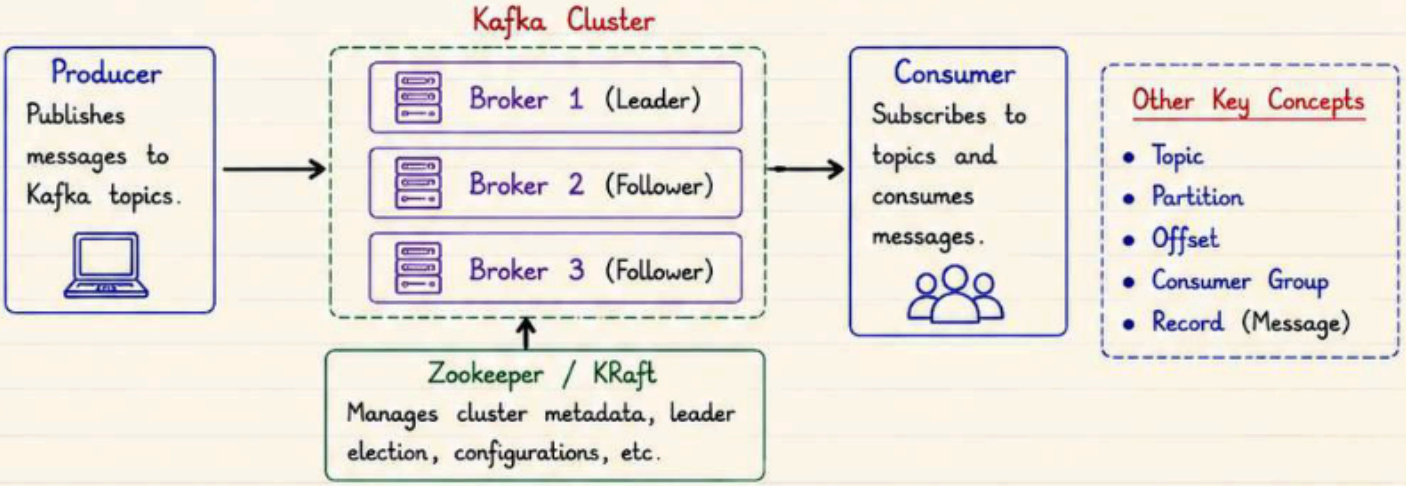
- ✓ High Throughput
- ✓ Scalable
- ✓ Fault Tolerant
- ✓ Durable
- ✓ Real-time Processing

Use Cases

- Real-time Analytics
- Log Aggregation
- Microservices Communication
- Event Sourcing
- Stream Processing

2. What are the main components of Kafka?

The main components of Kafka are:



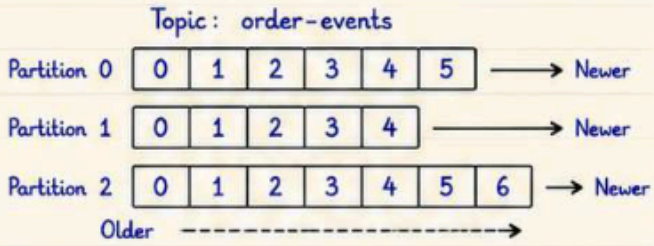
3. What is a Kafka topic?

A topic is a logical channel or category to which producers publish messages and from which consumers subscribe to consume.
Topics are identified by unique names.

Example:
user-signup, order-events, payment-updates, inventory-changes

Key Points

- Topics can have multiple partitions.
- Messages in a topic are immutable and ordered within a partition.
- Topics in Kafka cluster are replicated for fault tolerance.



4. What is a Kafka partition?

A partition is a subdivision of a topic. Each partition is an ordered, immutable sequence of records.
Each record is assigned a unique sequential ID called offset.
Partitions enable Kafka to scale horizontally and allow parallelism in data consumption.

Key Points

- Ordered within a partition.
- Immutable.
- Identified by partition ID.
- Stored across brokers.

TOP 30 KAFKA INTERVIEW QUESTIONS

5. Why does Kafka use partitions?

- Partitions allow Kafka to scale horizontally.
- They enable parallelism in both producers and consumers.
- Partitions provide high throughput and better performance.
- They also help in fault tolerance through replication.

Benefits

- ★ Scalability
- ★ Parallelism
- ★ High Throughput
- ★ Fault Tolerance
- ★ Better Performance

6. What is a Kafka broker?

A Kafka broker is a Kafka server that stores data, handles client requests, and manages topic partitions. A cluster consists of one or more brokers.

Broker Responsibilities

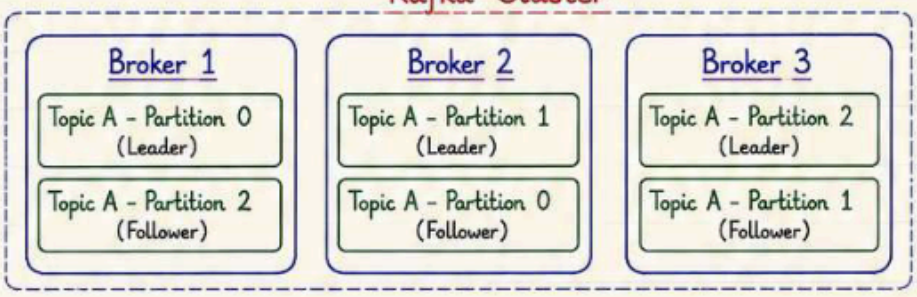
- Store messages
- Handle read/write requests
- Manage partitions
- Replicate data
- Coordinate with other brokers

7. What is a Kafka cluster?

A Kafka cluster is a group of one or more brokers working together to provide fault tolerance, high availability, and scalability.

Key Points

- Brokers work together as a cluster.
- Data is replicated across brokers.
- If one broker fails, others continue serving data.



8. What is the difference between a topic and a partition?

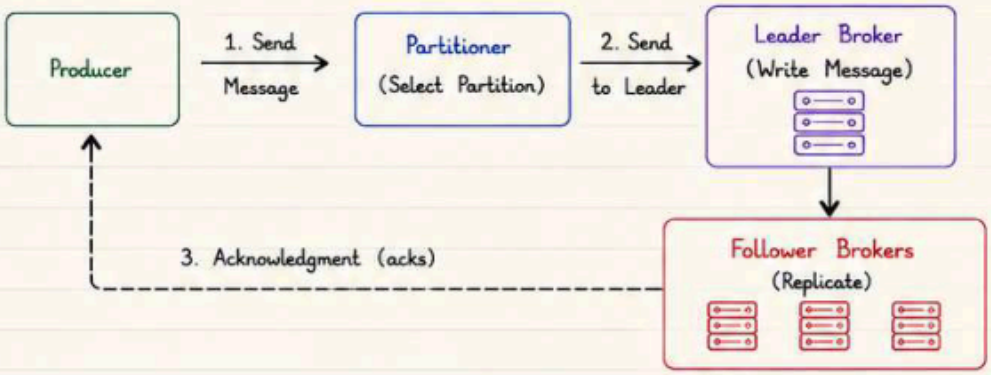
Topic	Basis	Partition
A topic is a logical channel or category to which messages are published.	Definition	A partition is a subdivision of a topic.
A topic can have one or more partitions.	Relation	A partition belongs to a topic.
It acts as the namespace for messages.	Purpose	It stores the actual messages in an ordered sequence.
Consumers subscribe to a topic.	Consumption	Consumers consume from partitions.
Example: order-events	Example	Example: order-events-0, order-events-1, order-events-2

Remember: A topic is like a folder, and partitions are like files inside that folder.

TOP 30 KAFKA INTERVIEW QUESTIONS

9. How does a Kafka producer send messages?

A producer publishes messages to a Kafka topic. The message is sent to the leader broker of the selected partition. The broker then replicates it to follower brokers.



Key Points

- Producer chooses partition and sends to leader.
- Leader writes and replicates to followers.
- Acknowledgment is sent back based on acks setting.

10. What is a producer acknowledgment (acks)?

Acknowledgment (acks) is the confirmation from Kafka brokers to the producer that a message has been received.

acks Value	Description	Durability	Performance
acks = 0	No acknowledgment from broker.	Low	High
acks = 1	Leader broker acknowledges after write.	Medium	Medium
acks = all	All in-sync replicas acknowledge.	High	Low

Key Points

- Controls durability vs performance.
- Higher acks = more durability, lower performance.
- Lower acks = higher performance, lower durability.

11. What is the difference between acks = 0, acks = 1, and acks = all?

acks = 0	acks = 1	acks = all (or -1)
• Producer does not wait for any acknowledgment. • Message may be lost if leader fails. • Highest throughput. • Use case: metrics, logs where some loss is acceptable.	• Leader acknowledges once message is written. • Message may be lost if leader fails before replication. • Balance between performance and safety. • Use case: default setting for many applications.	• All in-sync replicas acknowledge. • Message is safe even if some brokers fail. • Lowest throughput. • Use case: critical data, financial transactions.

☆ Note: acks = all == -1

12. What is a producer partitioning strategy?

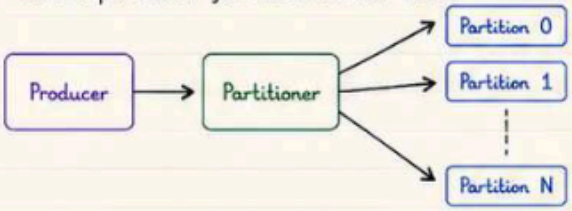
Partitioning strategy decides to which partition a message is sent when producing records to a topic.

Strategies:

- ➡ RoundRobinPartitioner (default): Distributes messages evenly across all partitions.
- ➡ KeyBasedPartitioner: If a key is provided, same key always goes to the same partition (hash based).
- ➡ StickyPartitioner (new default): Batches messages and sticks to the same partition for a period for better performance.
- ➡ Custom Partitioner: User-defined logic.

Key Points

- Key-based ensures ordering by key,
- Round-robin balances load.
- Sticky partitioner improves throughput by reducing partition switching.
- Custom partitioner for advanced use cases.



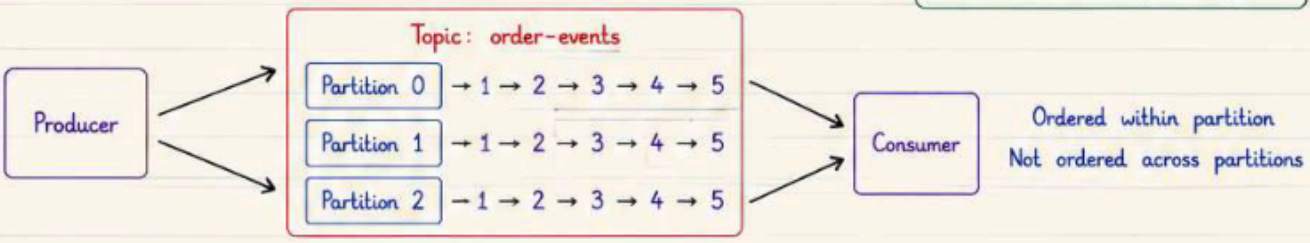
TOP 30 KAFKA INTERVIEW QUESTIONS

13. How does Kafka guarantee message ordering?

- Kafka guarantees message ordering at the partition level.
- Messages within the same partition are ordered.
- If messages are sent to different partitions, no ordering is guaranteed.
- Consumers must read from the same partition to get ordered messages.

Key Points

- Ordering is per partition.
- Use same key to ensure messages go to same partition.
- Do not increase partitions if strict ordering is required.



14. What is batching in Kafka?

Batching is the process of sending multiple messages together in a single request to Kafka broker.
It reduces the number of requests and improves throughput.

Key Points

- Improves performance.
- Reduces network overhead.
- Controlled by batch.size and linger.ms.



15. What is linger.ms and how does it affect performance?

linger.ms is the time (in ms) the producer waits before sending a batch even if the batch is not full.
It helps in collecting more messages in a batch which increases throughput but may add a small delay.

linger.ms	Batch Size	Throughput	Latency
Low (0-10 ms)	Small	Lower	Low
Medium (10-100 ms)	Medium	Medium	Medium
High (100+ ms)	Large	High	High

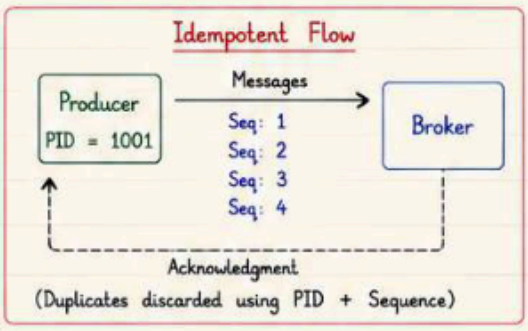
Key Points

- Higher linger.ms → bigger batches → higher throughput but more latency.
- Lower linger.ms → lower latency but smaller batches.
- Default: 0 ms.

16. What is idempotent message delivery in Kafka?

Idempotent producer ensures that messages are not duplicated even if retries happen.
Kafka assigns a Producer ID (PID) and a sequence number to each message. Broker uses them to detect and discard duplicates.
Benefits:

- Prevents duplicate messages on retries.
- Requires enabling idempotence and acks = all.



TOP 30 KAFKA INTERVIEW QUESTIONS

17. What is a Kafka consumer?

A Kafka consumer is an application that subscribes to one or more topics and reads messages published to those topics. It processes the messages and keeps track of its position (offset) in each partition.

Key Points

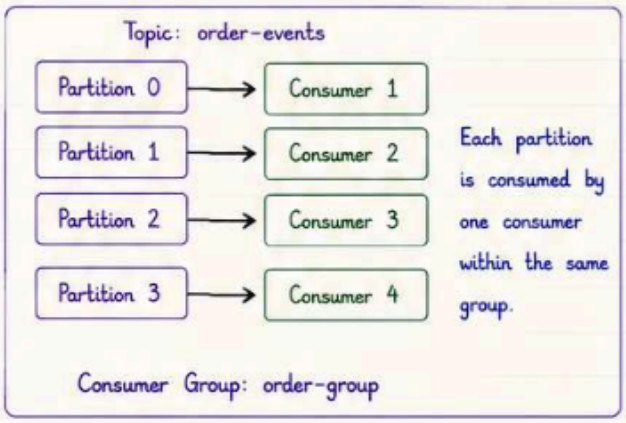
- Consumes messages from topics.
- Tracks its position using offsets.
- Can be part of a consumer group.
- Multiple consumers can read in parallel.

18. What is a consumer group?

A consumer group is a set of consumers that work together to consume messages from a topic (or multiple topics). Kafka ensures that each partition is consumed by only one consumer within the group.

Key Points

- Consumers in a group share the work.
- Each partition is assigned to only one consumer in the group.
- Consumers in different groups can read the same messages.

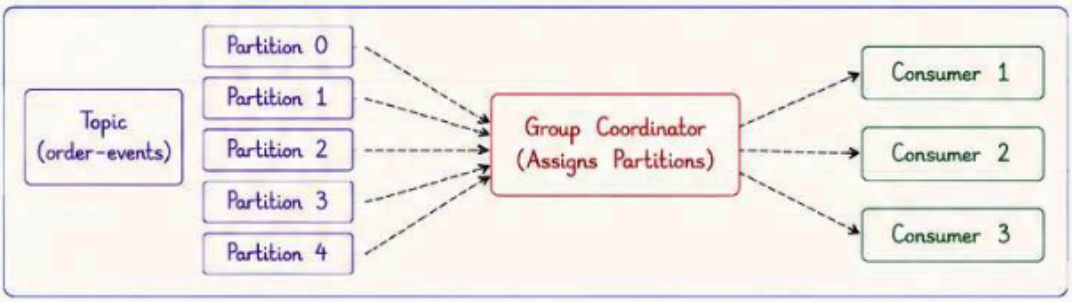


19. How are partitions assigned to consumers?

Kafka uses a group coordinator to assign partitions to consumers in a group. The default assignment strategy is RangeAssignor (earlier) and now mostly CooperativeStickyAssignor. Assignment happens during a rebalance.

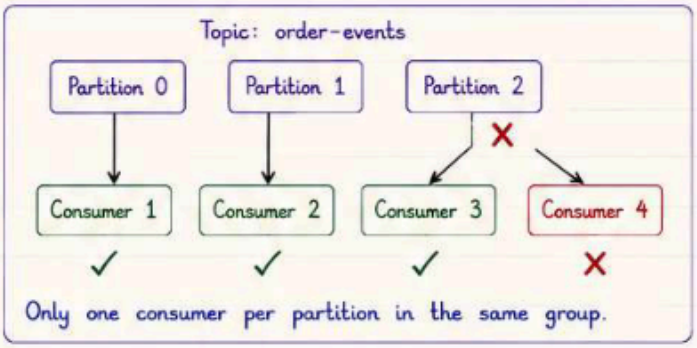
Key Points

- Group coordinator manages assignment.
- Default: RangeAssignor / CooperativeStickyAssignor.
- Assignment happens during rebalance.



20. Can two consumers in the same group consume the same partition?

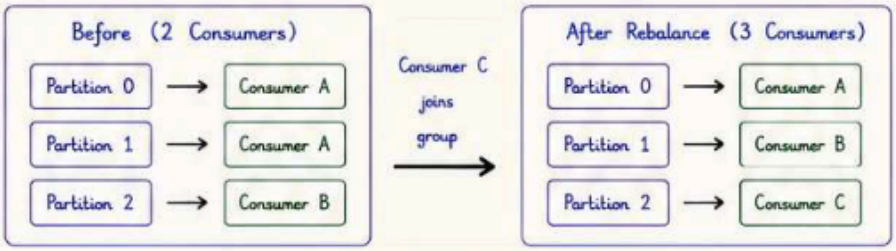
No. In the same consumer group, a partition is assigned to only one consumer. This ensures that messages in a partition are processed by a single consumer for ordering and consistency.



TOP 30 KAFKA INTERVIEW QUESTIONS

21. What happens when a consumer joins or leaves a consumer group?

When a consumer joins or leaves a group, a rebalance is triggered. The group coordinator reassigns partitions among the remaining consumers to maintain balance.



Key Points

- Join/leave triggers a rebalance.
- Partitions are redistributed among active consumers.
- Short pause may happen during rebalance.
- Frequent rebalances can impact performance.

22. What is a consumer rebalance?

A consumer rebalance is the process of reassigning partitions to consumers in a group when there is a change in the group membership or when partition subscriptions change.

Key Points

- Triggered by group changes or subscription changes.
- Ensures fair distribution of partitions.
- Consumers stop consuming briefly during rebalance.
- Controlled by group coordinator.

Reasons for Rebalance

- Consumer joins the group.
- Consumer leaves or crashes.
- Membership timeout expires.
- Partitions are added/removed from topic.
- Subscription (topics) changes.

1. Detect Change — Coordinator detects change in group membership.

→ **2. Stop The World** — All consumers stop consuming.

→ **3. Reassign Partitions** — Coordinator reassigns partitions to consumers. Consumers resume.

Total time = Rebalance Duration

23. What is a consumer offset?

An offset is a unique number assigned to each message in a partition. It represents the position of the message in that partition.

Topic: order-events Partition 0

Messages	M0	M1	M2	M3	M4	M5	M6	M7	...
Offsets	0	1	2	3	4	5	6	7	...

Key Points

- Offsets are per partition.
- Stored by consumer or external system.
- Enable tracking and resuming.
- Committed offsets = last processed.

24. What is the difference between auto commit and manual offset commit?

	Auto Commit	Manual Commit
Definition	Offsets are committed automatically by the consumer.	Offsets are committed explicitly by the application.
When committed	At fixed interval (auto.commit.interval.ms) in background.	When commitSync() or commitAsync() is called.
Reliability	Lower reliability, may lose some processed messages.	Higher reliability, commit after successful processing.
Control	Less control.	Full control over when to commit.
Use case	Use when occasional message loss is acceptable.	Use when at-least-once processing is required.

Key Points

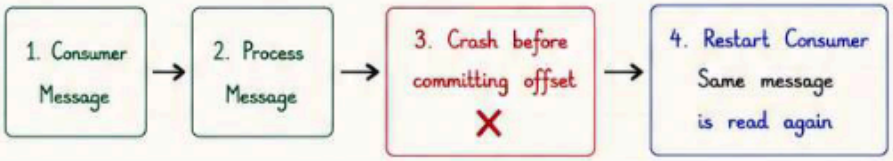
- Auto commit = simple but less reliable.
- Manual commit = more control and reliability.
- Most real-world apps use manual commit.

TOP 30 KAFKA INTERVIEW QUESTIONS

25. What happens if a Kafka consumer crashes after processing a message but before committing its offset?

If the consumer crashes after processing a message but before committing the offset, the offset is not saved. When the consumer restarts, it will read the same message again (from the last committed offset).

This can lead to duplicate processing (at-least-once delivery).



Key Points

- Offset not committed.
- On restart, message will be reprocessed.
- Leads to duplicate processing.
- To avoid duplicates, use idempotent processing or exactly-once semantics.

26. How does Kafka provide at-most-once, at-least-once, and exactly-once semantics?

Semantics	How it works	Guarantee
At-most-once	Commit offset before processing the message.	A message is delivered at most once. (May lose messages)
At-least-once	Commit offset after processing the message.	A message is delivered at least once. (May process duplicates)
Exactly-once	• Idempotent producer • Transactions (read-process-write) • EOS enabled consumers	A message is delivered exactly once. (No loss, no duplicates)

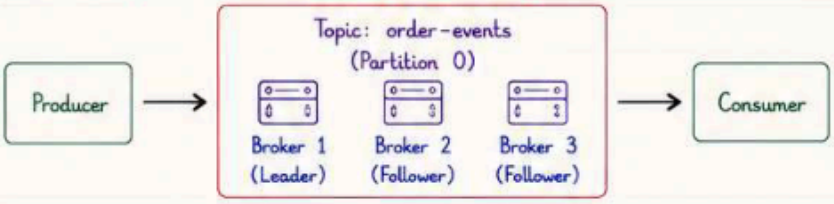
Key Points

- At-most-once → lower latency, some risk of loss.
- At-least-once → safe, but duplicates possible.
- Exactly-once → relies on idempotence + transactions.

27. What is replication in Kafka?

Replication is the process of copying partition data to multiple brokers.

It provides fault tolerance and high availability.

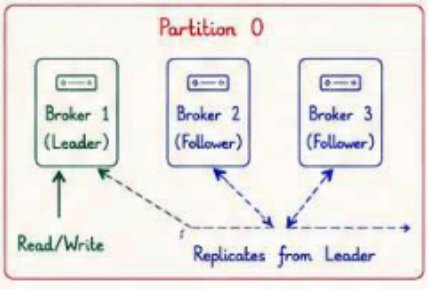


Key Points

- Replication factor defines number of copies.
- Default replication factor = 3.
- Improves fault tolerance and availability.
- Helps in data durability.

28. What are leader and follower replicas?

- Leader Replica: One replica is elected as the leader. It handles all read and write requests for the partition.
- Follower Replicas: Other replicas are called followers. They replicate data from the leader and stay in sync.



Key Points

- Leader handles reads/writes.
- Followers replicate from leader.
- Only one leader per partition.

TOP 30 KAFKA INTERVIEW QUESTIONS

29. How does Kafka handle data retention?

Kafka retains data for a configurable period of time or until the log size reaches a certain limit.
After retention time or size is reached, old messages are deleted to free up space.

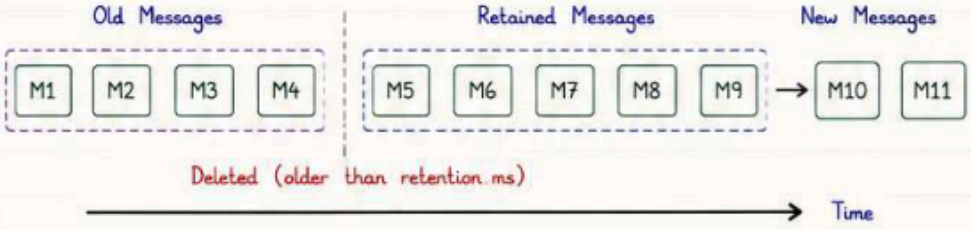
Key Points

- Retention is configured per topic.
- Based on time, size, or both.
- Deletion happens in the background.
- Does not affect active consumers.
- Compaction can be used for key-value topics.

Retention Policies

Policy	Description	Use Case
retention.ms	Deletes messages older than configured time.	Time-based retention.
retention.bytes	Deletes messages when topic size exceeds configured limit.	Size-based retention.
Both	Messages are deleted if either time or size limit is reached.	Balanced retention.
Log Compaction	Keeps the latest value for each key and removes old ones.	Key-value deduplication.

How Retention Works (Time-based)



Example

Topic: orders
retention.ms = 7 days
retention.bytes = 10 GB

Kafka keeps messages for 7 days or until the topic size reaches 10 GB, whichever comes first.

30. What is Kafka exactly-once semantics and how is it achieved?

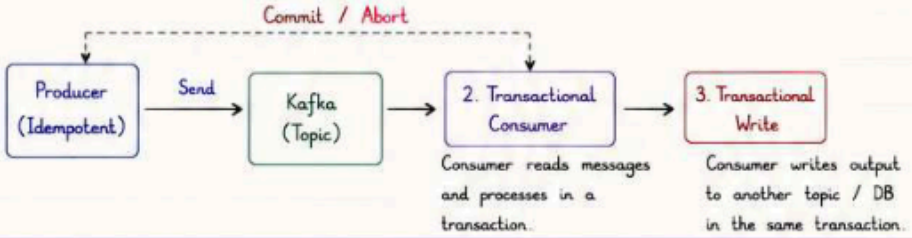
Exactly-once semantics means that each message is processed exactly once without duplicates or loss, even in case of failures. Kafka achieves this using idempotent producers, transactions, and EOS (Exactly Once Semantics) enabled consumers.

Key Points

- Ensures no duplicates and no message loss.
- Uses idempotent producer + transactions.
- Requires enabling EOS in both producer and consumer.
- Works with read-process-write workflows.

How Exactly-Once Semantics Works

1. Idempotent Producer
Producer assigns sequence number to each message to avoid duplicates.



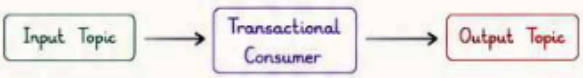
Example Scenario

Without Exactly-Once



- If consumer fails after processing but before committing, message may be processed again after restart.
- Leads to duplicate data in output.

With Exactly-Once



- Consumer uses transactions to write output.
- If failure happens, transaction is aborted.
- On restart, message is processed exactly once.
- No duplicates, no loss.

Requirements for Exactly-Once Semantics

- Producer: enable.idempotence = true
- Consumer: enable.auto.commit = false and isolation.level = read_committed
- Use transactions (producer.initTransactions(), beginTransaction(), commitTTransaction())
- Works with Kafka 0.11+